

Análisis y Diseño con Patrones

Práctica 1: Principios de diseño, patrones de análisis y arquitectónicos

Nos contratan para desarrollar un software para poner en contacto clientes con empresas de mudanzas. Para poder solicitar una mudanza, un cliente debe introducir su dni (id), nombre, apellidos, correo, y se suscribirá automáticamente a recibir ofertas. Si el usuario lo desea, podrá desuscribirse en cualquier momento pidiéndolo. En este caso, se registrará la razón por la que quiere desuscribirse.

Los clientes pueden solicitar una mudanza de un solo día o de varios días, dependiendo de lo que necesiten. La mudanza parte de un lugar de origen hasta un lugar destino, que debe tener las siguientes características: dirección, planta y si disponen de ascensor. Los clientes deberán marcar todos los tipos de objetos (item) que quieren transportar y si necesitan montaje o no. La mudanza será realizada por un solo operario de la empresa contratada. De cada empresa registra su NIF (taxId), nombre, precio por km y comisión que cobrará el operario.

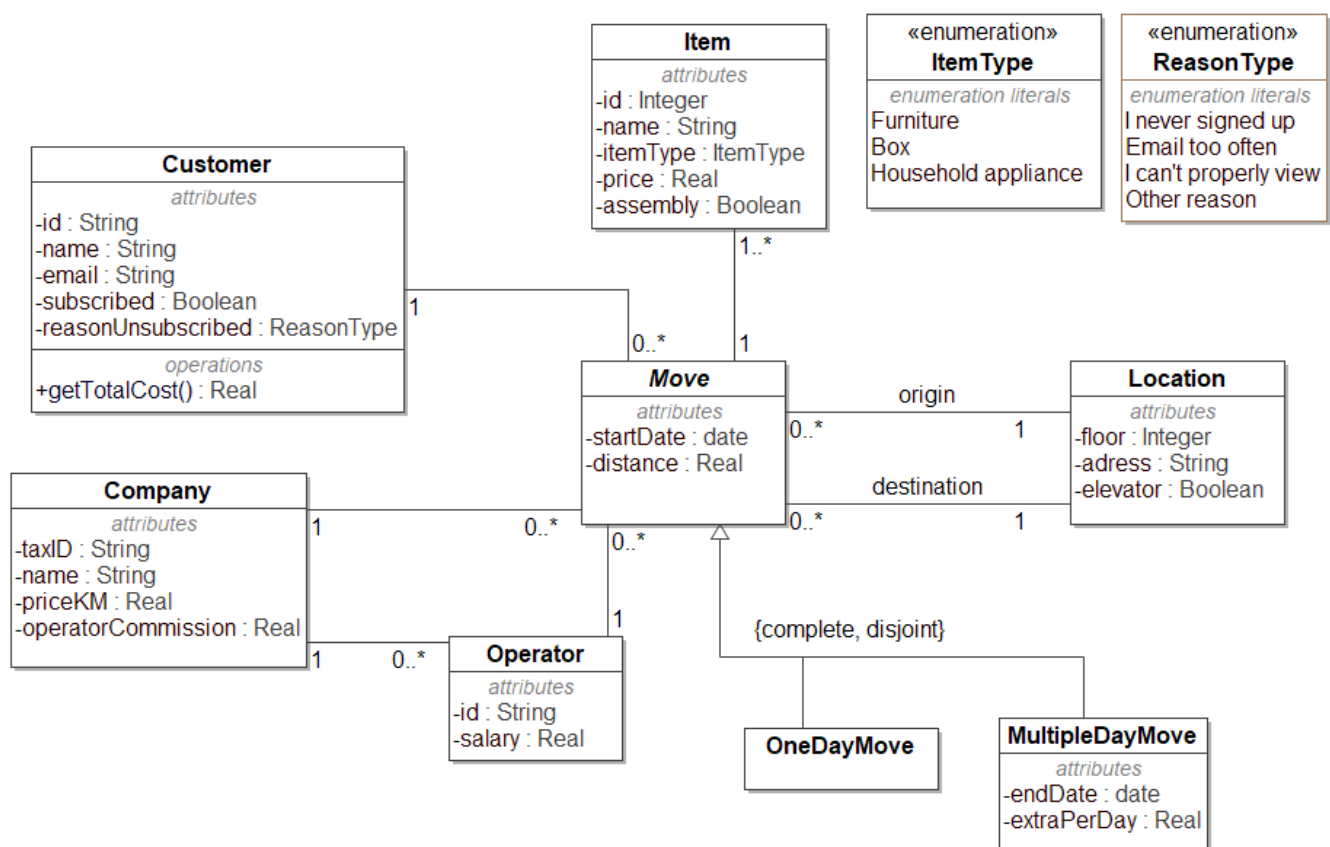
El precio de la mudanza dependerá de la distancia entre el origen y el destino (por el precio por km de la empresa), el precio de transportar cada objeto (que viene dado por el atributo precio del objeto y que ya viene definido por el sistema) y por el recargo de extra por día si la mudanza es de varios días.

Por ejemplo, si un cliente solicita una mudanza de dos días de Barcelona en Sant Cugat, con una distancia de 17,5 km, y quiere transportar 10 muebles, 2 electrodomésticos y 50 cajas, el precio de esta mudanza se calculará como la suma de: 17,5 kilómetros por el precio por km de la empresa, el precio de cada objeto y un día para el extra por día (el primer día no se cobra).

Este software es una parte de un sistema mayor. Obviaremos muchos elementos que un sistema real tendría. La razón de ello no es otra que la de hacer una práctica didáctica, y que la dificultad y dedicación prevista sean las deseadas.

Disponemos ya de un análisis previo que podemos utilizar como punto de partida, pero que habrá que corregir y mejorar. A continuación dispone del diagrama de clases para esta parte del software.

Diagrama de clases:



Claves de las clases:

- *Customer: id*
- *Operator: id*

- *Location: address*
- *Company: taxID*
- *Item: id*
- *Move: id(Customer) + originAddress + startDate*

Restricciones de integridad:

- La fecha de inicio de una mudanza debe ser anterior que la fecha final de MultipleDayMove.
- La dirección de origen no puede ser la misma que dirección destino.
- Un operario no puede tener mudanzas solapadas.
- Todos los integers y reales son números positivos.
- La empresa que hace la mudanza es la misma en la que trabaja el operario que participa.

Para realizar tareas de marketing, nos han pedido diseñar una operación para calcular el monto total del precio de las mudanzas (getTotalCost ()) que tiene un cliente. A continuación dispone del pseudocódigo de una posible implementación de este método:

```
public class Customer
{
    private String id;
    private String name;
    private String email;
    private Boolean subscribed;
    private ReasonType reasonUnsubscribed;
    private List<Move> moves;

    public Real getTotalCost()
    {
        Real total = 0;
        foreach (Move m in moves)
        {
            List<Item> items = m.getItems();
            foreach (Item i in items)
            {
                total += i.getPrice();
            }
            Company c = m.getCompany();
            total += m.getDistance() * c.getPriceKM();
        }
    }
}
```

```
        if(typeof(m)==MultipleDayMove) {
            Integer extraDays = m.getEndDate() - m.getStartDate();
            total += extraDays * m.getExtraPerDay();
        }
    }
    return total;
}

public class Item
{
    private Integer id;
    private String name;
    private ItemType itemType;
    private Real price;
    private Boolean assembly

    public Real getPrice()
    {
        return price;
    }
}

public class Company
{
    private String taxID;
    private String name;
    private Real priceKM;
    private Real operatorCommission;
    private List<Operator> operators;
    private List<Move> moves;

    public Real getPriceKM()
    {
        return priceKM;
    }
}
```

Ejercicio 1 (10%)

Revisamos el pseudocódigo de la operación `getTotalCost ()` de la clase `Customer` y nos preocupa que no se cumplan algunos de los principios de diseño. En concreto, se pide:

- a) Indica si este diseño satisface o no la Ley de Demeter y razona tu respuesta.
- b) Indica si este diseño satisface o no el principio de diseño Abierto-cerrado y razona tu respuesta.
- c) Indica si este diseño satisface o no el principio de diseño Bajo Acoplamiento y razona tu respuesta.

Solución:

- a) Esta solución no satisface la Ley de Demeter, ya que la clase `Customer`, en el método `getTotalCost`, utiliza las clases `Item`, `MultipleDayMove` y `Company` que no tiene directamente asociadas. Es decir, obtiene instancias de `Item`, `MultipleDayMove` y `Company` que no tiene asociadas, invoca métodos de las mismas y, por tanto, habla con desconocidos. En consecuencia, cualquier cambio en esta lógica propia de la clase `Item`, `MultipleDayMove` o `Company` podría afectar al código de la clase `Customer`.
- b) Esta solución viola el principio de diseño Abierto-Cerrado, ya que la clase `Customer` es consciente de que existe una subclase (`MultipleDayMove`) de `Move` e invoca una operación. Si, más adelante, ampliamos el sistema para admitir una nueva subclase de `Move` y la operación `getTotalCost` necesitaría invocar una operación de esta nueva subclase, deberíamos modificar el código de `Customer` añadiendo una nueva condición para tratar esta nueva subclase.
- c) Esta solución viola el principio de Bajo Acoplamiento. Como consecuencia de la no satisfacción de la Ley de Demeter, el acoplamiento de esta solución es alto. La clase `Customer` tiene conocimiento y, por tanto, acoplamiento, entre otras, con las clases `Item`, `MultipleDayMove` y `Company`.

Ejercicio 2 (20%)

Propón una solución alternativa completa (también en pseudocódigo del mismo estilo) que corrija los problemas de la operación `getTotalCost ()` de `Customer` que has detectado en el ejercicio anterior y que, por tanto, cumpla los principios violados identificados en el ejercicio 1. Modifica el diagrama de clases añadiendo todas las operaciones de tu solución.

Solución:

```
public class Customer
{
    private String dni;
    private String name;
    private String email;
    private Boolean subscribed;
    private ReasonType reasonUnsubscribed;
    private List<Move> moves;

    public Real getTotalCost()
    {
        Real total = 0;
        foreach (Move m in moves)
        {
            total += m.getTotalCostMove();
            // ahora Customer solo habla con Move
            // ahora la Ley de Demeter se cumple
            // ahora el acoplamiento es más bajo
            // el uso de esta operación polimórfica hace que se cumpla
            // el principio Abierto-Cerrado.
        }
        return total;
    }
}
```

```
public abstract class Move
{
    protected Date startDate;
    protected Real distance;
    protected Location origin;
    protected Location destination;
    protected List<Item> items;
    protected Operator operator;
    protected Company company;

    public Real getTotalCostMove()
    {
        Real total = 0;
        foreach (Item i in items)
        {
            total += i.getPrice();
        }
        total += distance * company.getPriceKM();
        total += getAdditionalCharges();
        return total;
    }

    protected abstract Real getAdditionalCharges();
    // operación abstracta que será implementada en cada subclase;
    // la decisión de cuáles son los cargos a sumar depende de la subclase
}

public class MultipleDayMove inherits Move
{
    private Date endDate;
    private Real extraPerDay;

    public override Real getAdditionalCharges()
    {
        Integer extraDays = endDate - startDate;
        return extraDays * extraPerDay;
    }
}
```

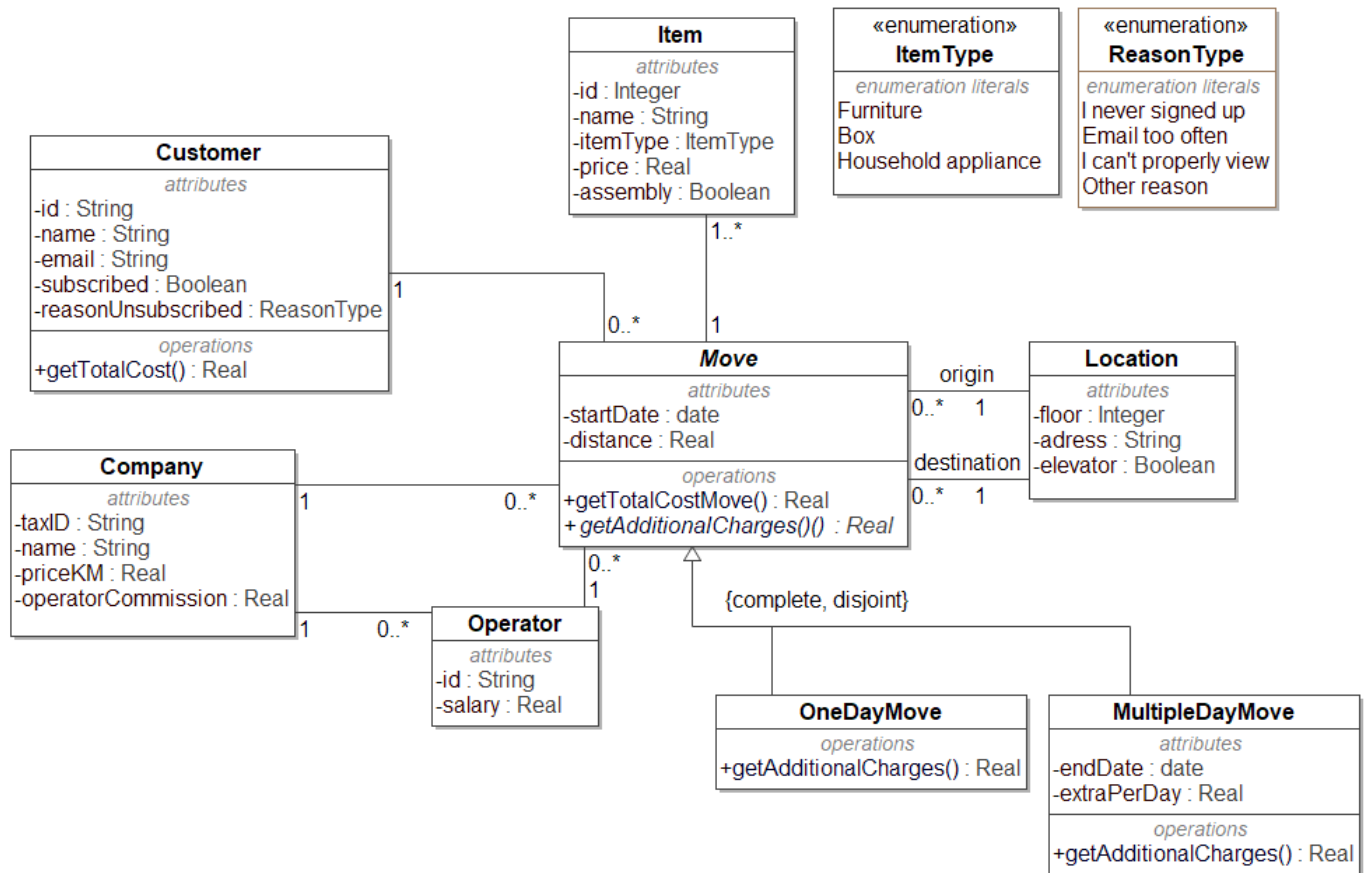
```
public class DayMove inherits Move
{
    public override Real getAdditionalCharges()
    {
        return 0;
    }
}
```

```
public class Item
{
    private Integer id;
    private String name;
    private ItemType itemType;
    private Real price;
    private Boolean assembly;

    public Real getPrice()
    {
        return price;
    }
}
```

```
public class Company
{
    private String taxID;
    private String name;
    private Real priceKM;
    private Real operatorCommission;
    private List<Operator> operators;
    private List<Move> moves;

    public Real getPriceKM()
    {
        return priceKM;
    }
}
```

Ejercicio 3 (30%)

Recientemente, las empresas de mudanzas se han dado cuenta de que tienen que cambiar el sueldo (salary) de los operarios más de lo que querrían dependiendo de la época. Es por ello que se ha decidido registrar los sueldos de los operarios durante el tiempo. De esta manera los operarios podrán ver el registro de su sueldo en períodos de tiempo diferentes.

Nos proponen buscar una solución que nos permita representar esta información en nuestro sistema. En concreto, se pide:

- a) Describe brevemente como solucionarías este problema. En caso de que propongas la aplicación de un patrón, explica qué y razona su conveniencia.
- b) Propone una solución detallada en forma de diagrama estático de análisis. Mostrar sólo las clases que cambien respecto al diagrama del enunciado y las restricciones de integridad que aparezcan.
- c) Indica cómo sería el pseudocódigo resultante de la operación `higherSalaryOperator()`: String de la clase `Company` que devuelve el id del operario con el sueldo más alto del mes actual. El sueldo mensual de un operario se calcula con su sueldo más la comisión (`operatorCommission`) por cada mudanza en la que ha participado durante el mes actual.

Nota: las mudanzas de más de un día se contabilizan al mes en el que empiezan. Por ejemplo, una mudanza que abarque del 30 de abril al 2 de mayo, se contabiliza como efectuada en el mes de abril.

Solución:

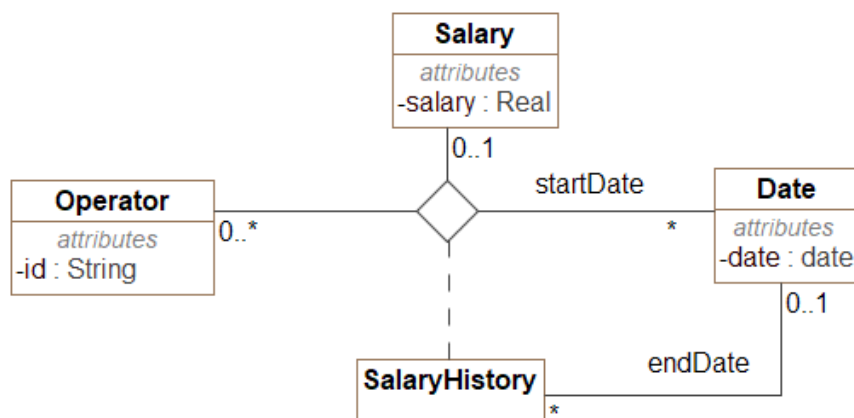
- a) Según el diagrama de clases del enunciado, sólo se registraba el sueldo actual de un operario. Para poder representar que un operario puede tener diferentes sueldos a lo largo del tiempo tenemos que utilizar el patrón Asociación Histórica.

Este patrón nos permite transformar el atributo `salary` en un histórico de los valores que ha tomado el atributo a lo largo del tiempo proporcionando el histórico de los sueldos de los operarios.

- b) Primero pasamos el atributo `Salary` a una clase y la asociamos a la clase `Operator`:



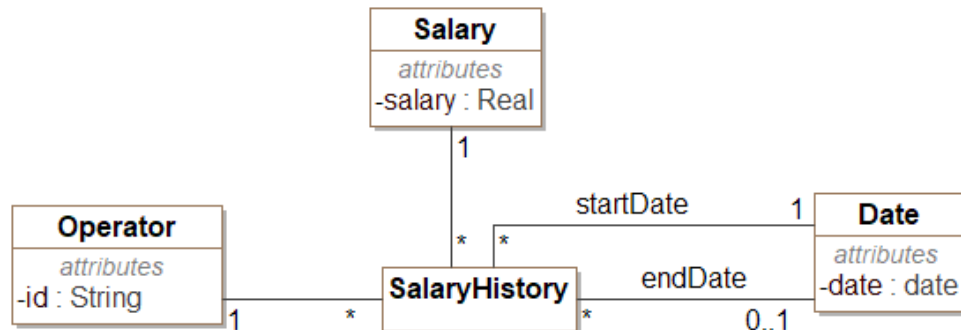
Una vez pasado el atributo a una clase, ya podemos aplicar el patrón asociación histórica. El resultado de la aplicación del patrón es:



Restricciones de integridad:

- La fecha final ha de ser superior a la data de inicio.
- Un operario no puede tener salarios solapados.

Los lenguajes de programación orientados a objetos actuales no permiten definir asociaciones ternarias, ni clases asociativas. Para poder trabajar con estos constructores, hay que aplicar una pequeña transformación en el diagrama de clases para obtener otro diagrama de clases que represente la misma semántica pero con constructores que los lenguajes de programación orientados a objetos permitan representar. Aplicando esta transformación obtendríamos:



Restricciones de integridad:

- Clave de SalaryHistory: idOperator + startDate
- Salary es un atributo opcional.

De hecho, como Date (fecha) se podría considerar como un tipo de datos, y los tipos de datos los pondríamos normalmente como atributos, esta solución la podemos transformar en:



c)

```

public class Company
{
    private String taxID;
    private String name;
    private Real priceKM;
    private Real operatorCommission;
    private List<Operator> operators;
    private List<Move> moves;
}
  
```

```
public String higherSalaryOperator()
{
    Real maxSalary = 0;
    foreach (Operator o in operators)
    {
        Real monthExtra = o.getNumMovesCurrentMonth() * operatorCommission;
        Real salary = o.getSalary() + monthExtra;
        if (salary > maxSalary) {
            maxSalary = salary;
            id = o.getID();
        }
    }
    return id;
}
```

```
public class Operator
{
    public String id;
    public Real salary;
    public List<Move> moves;

    public String getID()
    {
        return id;
    }

    public Real getSalary()
    {
        return Salary;
    }

    public Real getNumMovesCurrentMonth()
    {
        Real movesMonth = 0;
        foreach (Move m in moves)
        {
            Date d = m.getStartDate();
            Date today = now();
```

```
        if (d.getYear() == today.getYear() and d.getMonth() ==
today.getMonth()) { //comprobamos que sea el mismo año y mes
            movesMonth += 1;
        }
    }
    return movesMonth;
}
}

public abstract class Move
{
    protected Date startDate;
    protected Real distance;
    protected Location origin;
    protected Location destination;
    protected List<Item> items;
    protected Operator operator;
    protected Company company;

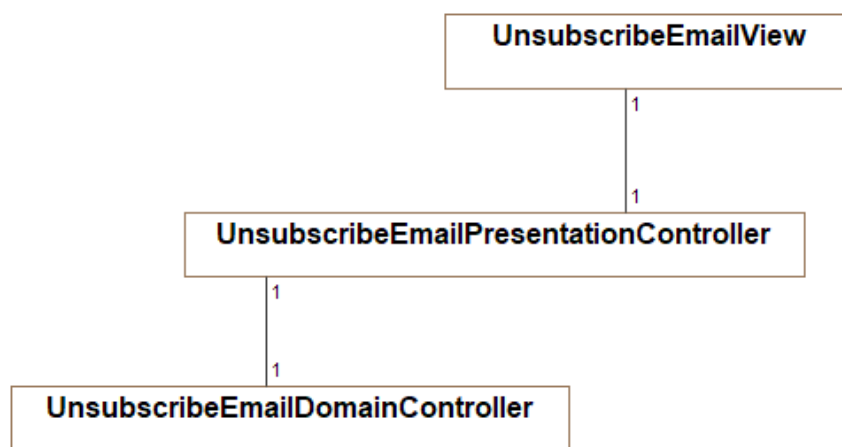
    public String getStartDate()
    {
        return startDate;
    }
}
```

Ejercicio 4 (10%)

Queremos definir la capa de presentación de nuestro sistema y para ello aplicamos el patrón de arquitectura Modelo, Vista y Controlador (MVC). Os proponemos un diagrama estático de análisis a medias donde se aplica el patrón MVC para el caso de uso Desuscripción correo. Este caso de uso permite al usuario indicar la opción de dar de baja su correo para no recibir publicidad de la empresa. En respuesta, el sistema le mostrará todos los motivos posibles para darse de baja. El usuario seleccionará el motivo por el que se quiere dar de baja y el sistema le dará de baja y le mostrará el mensaje "Unsubscribed successfully".

El diagrama tiene todas las clases necesarias. Se pide explicar qué representa cada clase (en el patrón MVC) y definir la firma de las operaciones desde que el usuario ve un panel de control con todas las acciones que puede hacer, en la que una de ellas es el opción desuscripción correo, hasta que el sistema da de baja el correo mostrando el mensaje "Unsubscribed successfully".

Para cada operación, debe especificar la clase a la que pertenecen, los parámetros, lo que devuelve y explicar qué hace.



Solución:

Aplicaremos el MVC en la capa de Presentación. La clase **UnsubscribeEmailView** representa la vista del patrón, la clase **UnsubscribeEmailPresentationController** representa el controlador y la clase **UnsubscribeEmailDomainController** representa el controlador de la capa de dominio.

A continuación se describen las operaciones de cada clase:

Operaciones de la clase **UnsubscribeEmailView**:

- **UnsubscribeEmailView::showOptions()** muestra las diferentes opciones correspondientes a los diferentes casos de uso (entre ellos la baja del correo) a la vista.

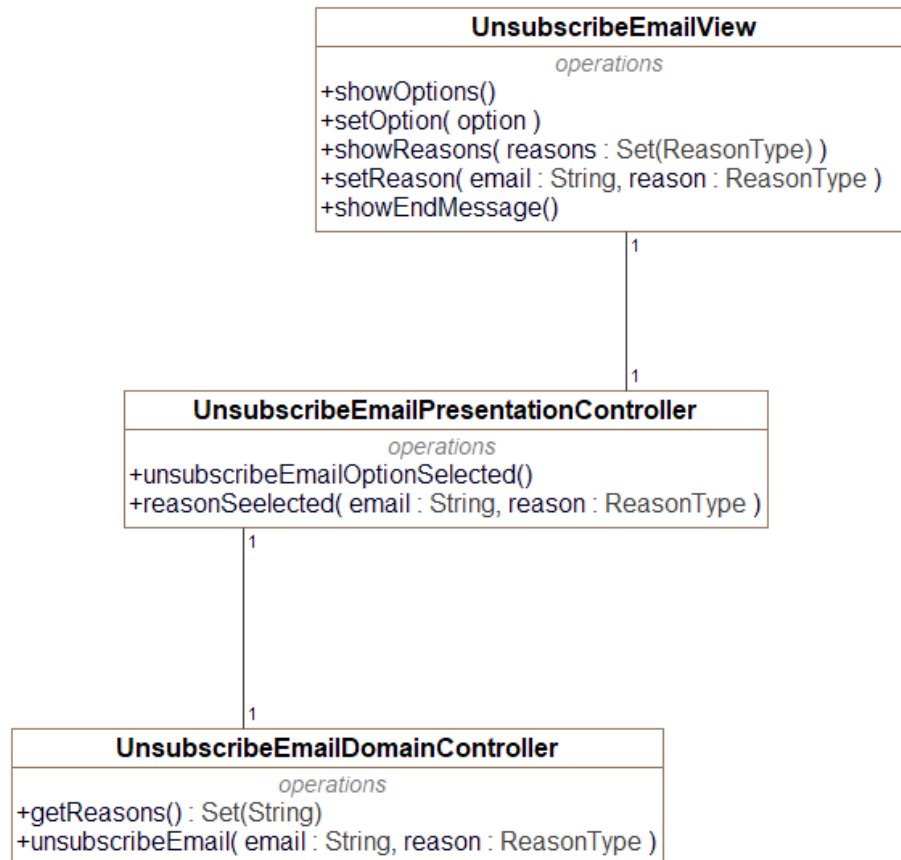
- `UnsubscribeEmailView::setOption(option)` registra el caso de uso seleccionado y lo notifica al controlador. Sólo si se quiere que la vista registre el caso de uso seleccionado.
- `UnsubscribeEmailView::showReasons(reasons: Set (ReasonType))` muestra la vista con un desplegable de los posibles motivos para darse de baja que hay almacenados en el sistema.
- `UnsubscribeEmailView::setReason(email:String, reason: ReasonType)` registra el correo y el motivo por el que se da de baja y lo notifica al controlador.
- `UnsubscribeEmailView::showEndMessage()` muestra la vista con el mensaje "Unsubscribed successfully".

Operaciones de la clase **UnsubscribeEmailPresentationController**:

- `UnsubscribeEmailPresentationController::unsubscribeEmailOptionSelected()` detecta que el usuario ha elegido la opción para dar de baja el correo e invoca a la operación.
- `UnsubscribeEmailPresentationController::reasonSelected(email: String, reason: ReasonType)` detecta que el usuario ha marcado el motivo para darse de baja e invoca a la operación.

Operaciones de la clase **UnsubscribeEmailDomainController**:

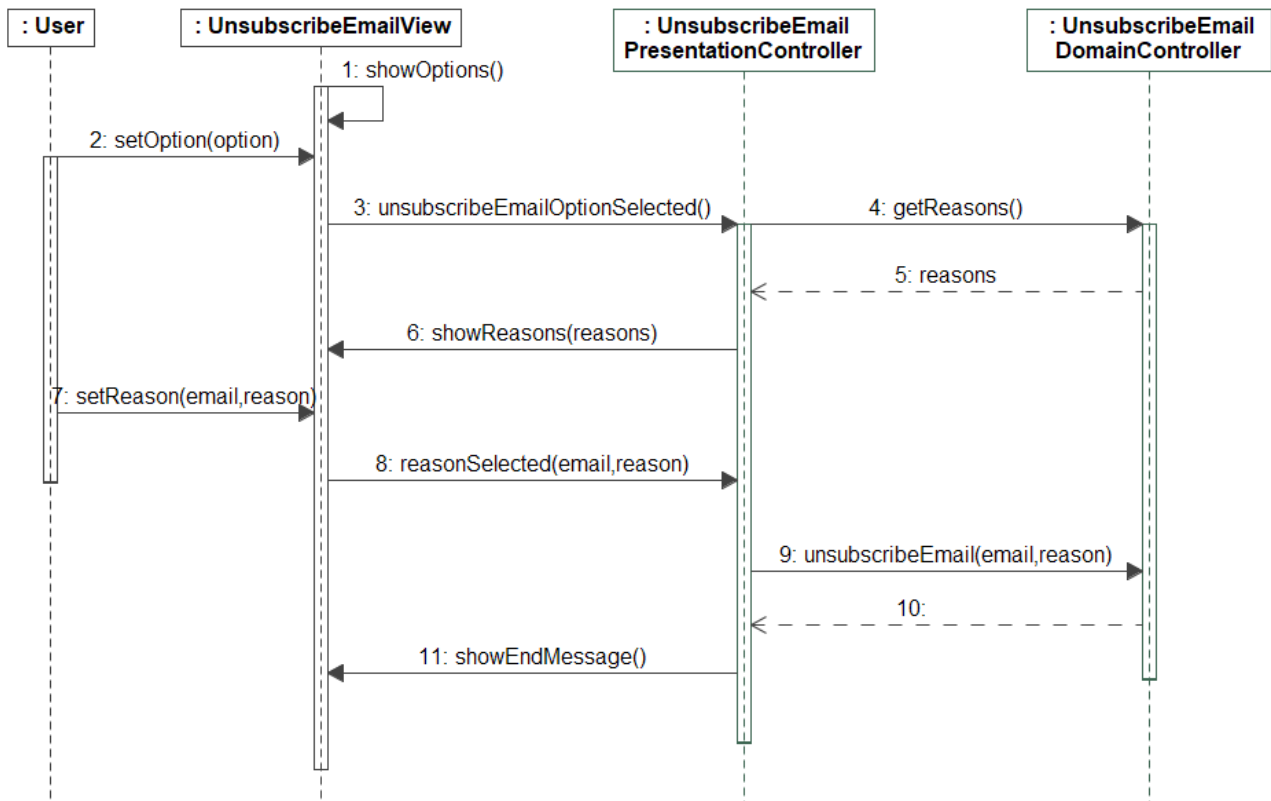
- `UnsubscribeEmailDomainController::getReasons(): Set(ReasonType)` obtiene las razones de darse de baja el correo que hay en el sistema.
- `UnsubscribeEmailDomainController::unsubscribeEmail(email: String, reason: ReasonType)` da de baja el correo y registra la razón.



Ejercicio 5 (10%)

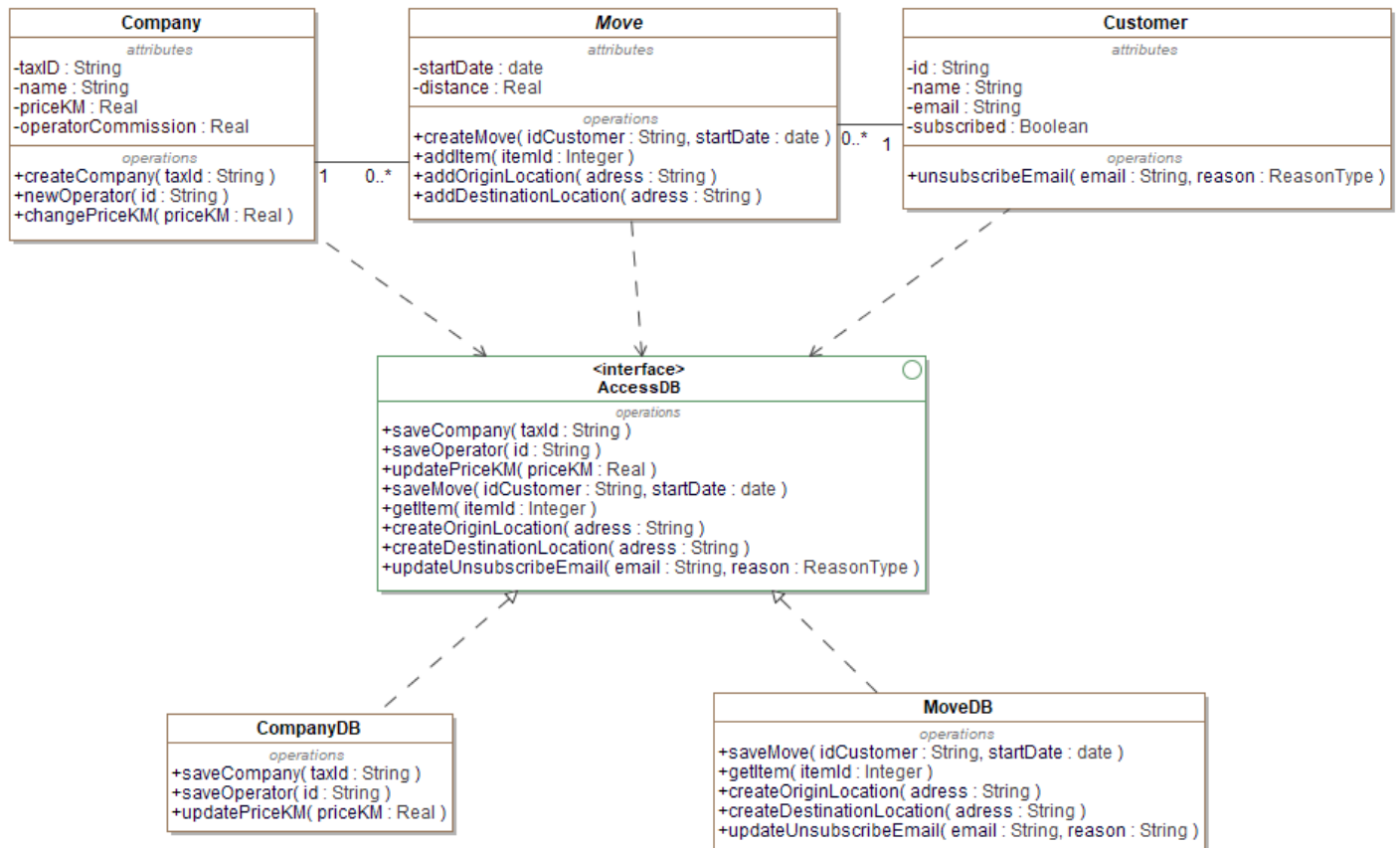
Proponed el diagrama de secuencia de la interacción desde el usuario hasta el controlador de dominio para el caso de uso Desuscripción correo, que acaba mostrando el mensaje "Unsubscribed successfully".

Solución:



Ejercicio 6 (10%)

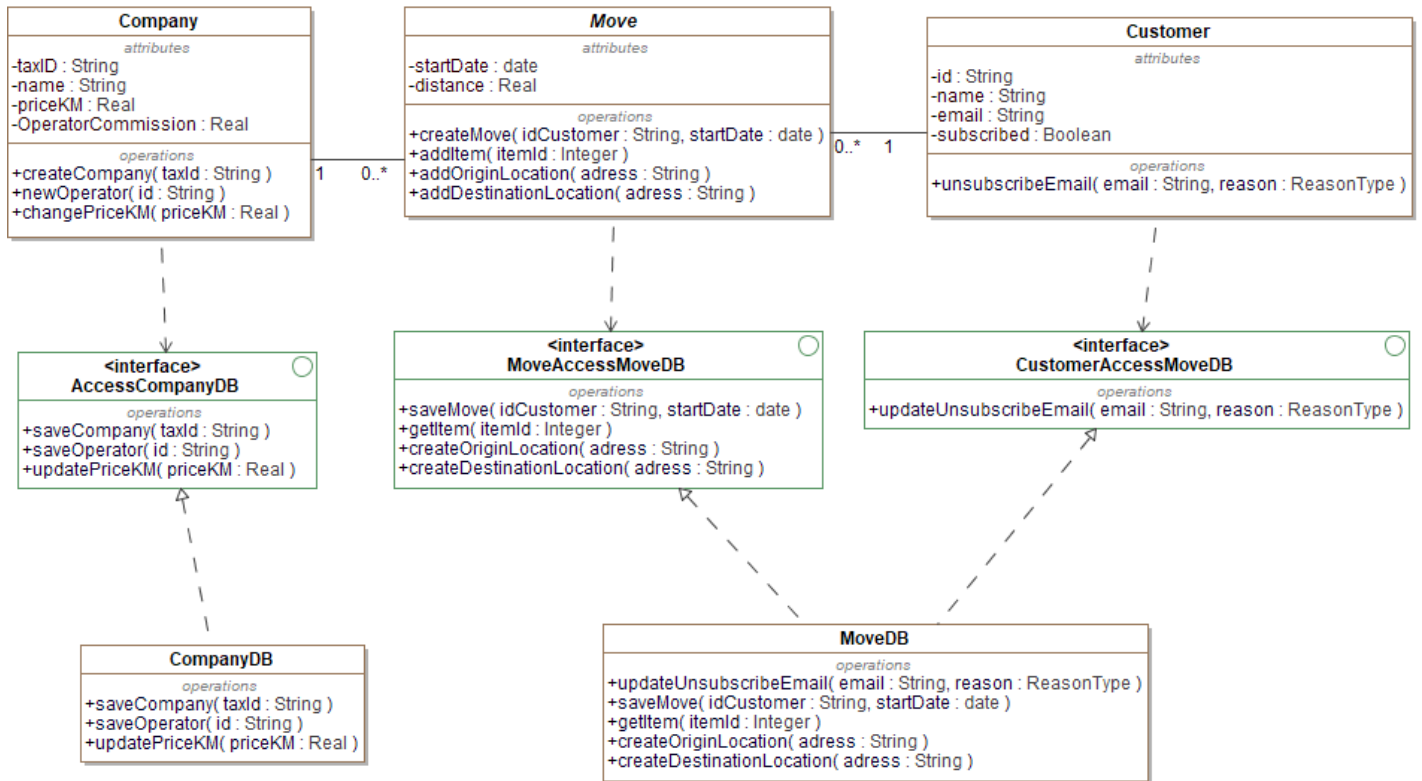
Seguimos diseñando la arquitectura del sistema y hemos decidido utilizar una arquitectura con 3 capas: presentación, dominio y servicios técnicos. Nos encontramos que la empresa tiene dos bases de datos que no quieren fusionar, una por la parte de las compañías y operarios y otra en relación con los clientes y las mudanzas. Hemos hecho un primer diseño de cómo se podrían comunicar las clases que ya tenemos con las bases de datos:



¿Qué principio de diseño está violando? Justifica tu respuesta. Propón un diagrama estático de análisis que solucione la violación del principio.

Solución:

El principio de diseño que se viola es el de Segregación de interfaces. Este principio nos dice que los clientes no deberían depender de operaciones que no utilizan. En lo que se presenta los objetos cliente **CompanyDB** depende de operaciones que no utiliza como todas las que van a parar a la base de datos de **MoveDB** y el objeto **MoveDB** depende de las operaciones **CompanyDB** que tampoco utiliza. El diagrama estático de análisis que soluciona la violación de este principio es:



Ejercicio 7 (10%)

Para acabar de diseñar la arquitectura del sistema, como ya hemos decidido utilizar una arquitectura con 3 capas: presentación, dominio y servicios técnicos. Utilizaremos las tres capas clásicas de muchos sistemas de información:

1. Presentación
2. Dominio
3. Servicios técnicos

Queremos reflexionar sobre a qué capa pertenece cada una de las clases que han ido surgiendo en el enunciado (y la solución que has elaborado hasta ahora):

- Clases del primer diagrama: **Item**, **Move**, **OneDayMove**, **MultipleDayMove**, **Location**, **Customer**, **Operator** y **Company**

- Clases del segundo diagrama (Ejercicio 4): UnsubscribeEmailView, UnsubscribeEmailPresentationController, UnsubscribeEmailDomainController.
- Clases del tercer diagrama (Ejercicio 6): AccessDB (interfaz), CompanyDB, MoveDB
- Posibles clases y / o interfaces introducidas / cambiadas en tu solución del Ejercicio 6.

- a) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de dominio. Razona tu respuesta.
- b) Indica qué clases (e interfaces, si las hay) pertenecen a la capa de servicios técnicos. Razona tu respuesta.

Solución:

a)

Las clases de la capa de dominio es donde reside la lógica de negocio, donde trabajan el problema que nuestro sistema resuelve.

Esto incluye, naturalmente, las clases, las instancias de las cuales representan objetos, documentos o valores del mundo real, como una mudanza, cliente, localización, objeto, operador y compañía:

- Item, Move, OneDayMove, MultipleDayMove, Location, Customer, Operador y Company y los tipos de los atributos al primer diagrama

También incluye las clases que ofrecen operaciones del nivel del dominio para ser usadas por la capa de presentación, que podemos llamar controladores de dominio:

- UnsubscribeEmailDomainController

Finalmente, por segregación de interfaz, hemos añadido clases que representan el conjunto de objetos de dominio que el sistema conoce y que eventualmente serán implementadas por la capa de servicios técnicos para ofrecer persistencia:

- AccessCompanyDB y AccessMoveDB (de la solución del ejercicio 6)

b)

La capa de servicios técnicos tiene el nivel de abstracción de la infraestructura sobre la que corre nuestro sistema (ficheros, puertos, etc.) o el nivel de abstracción de servicios externos como los de las bases de datos. En nuestro caso, por tanto, incluye aquellas clases que implementan el acceso a una base de datos relacional para recuperar datos persistentes:

- *CompanyDB*, *MoveDB*